TUS
Technological University of the Shannon:
Midlands Midwest
Ollscoil Teicneolaíochta na Sionainne:
Lár Tíre Iarthar Láir

Weverton de Souza Castanho
Continuous Build and Delivery
21.APRIL.2026

Stock Options Time Series Predictor

Continuous Build and Delivery Pipeline

https://github.com/wevertonsc/TUS_STP_CBD_SEM2_4_2026

Abstract

The Continuous Deployment and Continuous Integration discipline provided an opportunity to understand how difficult this type of activity is in the daily work of teams that use it in production environments. Details of configuration and integration of all stages of development, deployment strategies, and infrastructure configuration working together demonstrate how each part needs to integrate with the others so that everything can run perfectly. Often, these delivery and deployment processes are performed hundreds and thousands of times in large corporations, and yet when we open our computers or cell phones, in most cases these services are available and working without problems, but we don't have a clear idea of the amount of effort and time invested in keeping all this running as if it were something simple. According to Humble and Farley (2010), Continuous Delivery is defined as a discipline in which software is built in such a way that it can be released to production at any time.

Introduction

Delving a little deeper into the concepts of Continuous Delivery (CD), it expands on CI, ensuring that software can be reliably released to production at any time. This requires the system to go through an automated pipeline that includes compilation, testing, and packaging, culminating in a deployable artifact without human intervention, and all in a very successful way!

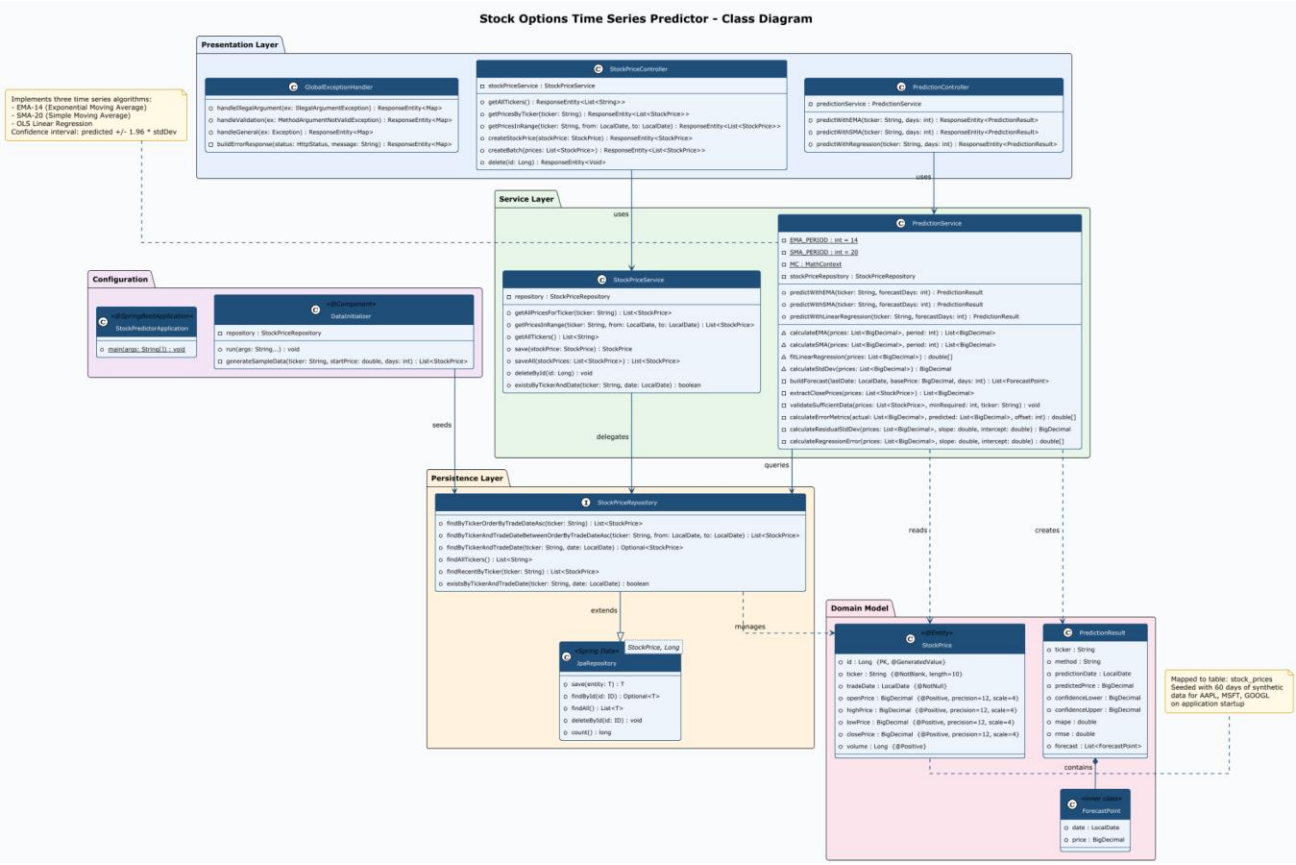
Dimension	Continuous Integration	Continuous Delivery
Primary Goal	Detect integration defects early	Ensure releasability at all times
Trigger	Code commit to shared branch	Successful CI pipeline execution
Artefact	Compiled and tested build	Deployable package (JAR, image)
Deployment	Not necessarily automated	Automated up to staging or production
Human approval	Not required	Optional gate before production

According to the principles described by Forsgren, Humble and Kim (2018), modern CI/CD practices have several fundamental principles, among which we can include trunk-based development, automated quality gates, shift-left testing and infrastructure as code.

The key distinctions between the two Continuous Integration and Continuous Delivery:

Category	Tool Used	Rationale
Version Control	GitHub	Industry-standard distributed VCS with pull request workflow and Actions integration
Build Automation	Maven 3.9	Mature dependency management and lifecycle management for Java projects
CI Server	Jenkins / GitHub Actions	Jenkins for on-premise flexibility; Actions for cloud-native pipelines-as-code

Category	Tool Used	Rationale
Static Analysis	SonarQube	Comprehensive quality gate enforcement with coverage, duplication, and bug detection
Test Coverage	JaCoCo	Native JVM byte-code instrumentation with Maven plugin integration
Containerisation	Docker	OCI-compliant image standard supported by all major cloud providers
Deployment Automation	Ansible	Agentless, idempotent infrastructure automation using YAML playbooks



User Stories

The following user stories capture the functional requirements of the Stock Options Time Series Predictor microservice. Each story is written in the Who/What/Why format with associated acceptance criteria.

US-001: EMA Price Prediction

Story	As a quantitative analyst, I want to request an Exponential Moving Average (EMA-14) prediction for a given stock ticker so that I can identify short-term momentum trends.
Acceptance Criteria	The endpoint GET /api/v1/predict/ema/{ticker} returns HTTP 200 with a JSON body containing predictedPrice, confidenceLower, confidenceUpper, mape, rmse, and a forecast array. A minimum of 14 historical records must exist for the ticker. If insufficient data exists, the API returns HTTP 400 with a descriptive error message.

US-002: SMA Price Prediction

Story	As a portfolio manager, I want to obtain a Simple Moving Average (SMA-20) forecast for a stock so that I can compare it against the EMA prediction to assess trend reliability.
Acceptance Criteria	The endpoint GET /api/v1/predict/sma/{ticker} returns a valid PredictionResult. The method field in the response identifies SMA. Confidence bounds are always present and lower < upper. Forecast array length matches the 'days' request parameter.

US-003: Linear Regression Trend Forecast

Story	As a data scientist, I want to retrieve an OLS linear regression forecast so that I can evaluate the long-term price trend of a stock option.
Acceptance Criteria	The endpoint GET /api/v1/predict/regression/{ticker} returns a forecast computed via OLS. Forecast points are ordered chronologically. MAPE and RMSE metrics are non-negative numbers. At least 10 historical records are required.

US-004: Historical Data Ingestion

Story	As a data engineer, I want to ingest historical stock price records via a REST API so that the prediction engine has sufficient training data.
Acceptance Criteria	POST /api/v1/stocks returns HTTP 201 with the persisted entity. POST /api/v1/stocks/batch supports bulk insertion. All price fields (open, high, low, close) must be positive values. Missing required fields return HTTP 400.

US-005: Ticker Discovery

Story	As a consumer of the API, I want to discover which ticker symbols are available so that I can make valid prediction requests.
Acceptance Criteria	GET /api/v1/stocks/tickers returns an alphabetically ordered JSON array of available ticker strings. The endpoint returns HTTP 200 even when the list is empty.

US-006: Operational Health Monitoring

Story	As a DevOps engineer, I want a health check endpoint so that the container orchestrator can verify the service is running correctly.

Acceptance Criteria	GET /actuator/health returns HTTP 200 with JSON body containing status: UP when the service is operational. The Docker HEALTHCHECK directive uses this endpoint.

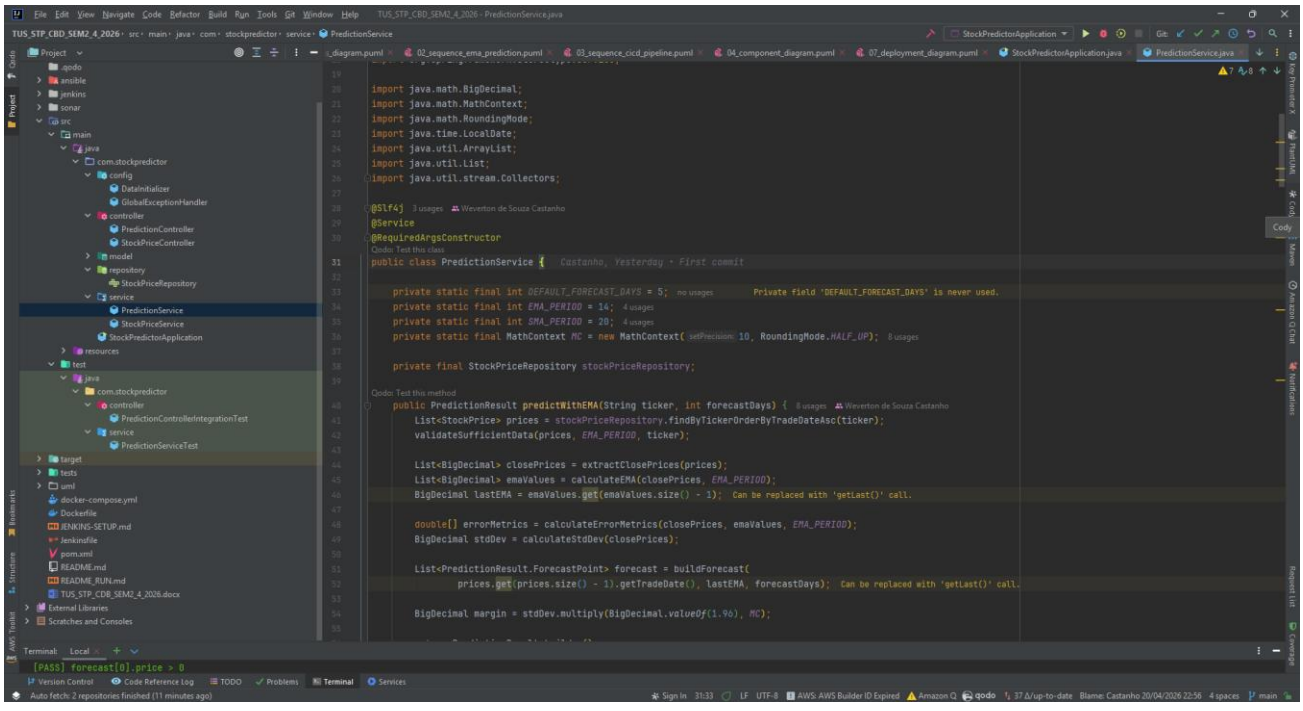
Application Architecture

The application implemented, **Stock Options Time Series Predictor**, for the assessment is based on a REST microservice developed in **Java 21** using **Spring Boot 3.4.4** that aims to exploit the predictive resource of time series for stock option price prediction. Given a stock option ticker (e.g., **AAPL**, **MSFT**, **GOOGL**) plus the number of days the user wants to predict, the system starts the forecast calculation, based on 3 algorithms as follows:

- **EMA-14** - Exponential Moving Average with a 14-day window. Gives more weight to recent observations, becoming more reactive to trend changes.
- **SMA-20** - Simple Moving Average with a 20-day window. Smooths out short-term fluctuations by calculating the simple arithmetic mean of the last 20 prices.
- **OLS Linear Regression** - Ordinary Least Squares. Fits a regression line to historical data and extrapolates to subsequent days, capturing long-term linear trends.

Implementation

The application follows a classic three-layer architecture of the Spring ecosystem:



The presentation layer exposes documented REST endpoints via Swagger UI. The PredictionController serves three prediction routes, and the StockPriceController generates a complete CRUD from historical data, with input validation and global error handling performed by the GlobalExceptionHandler.

The service layer is responsible for all the business logic of the web services, with PredictionService implementing the three algorithms directly in pure Java, without external machine learning dependencies, and StockPriceService orchestrating data operations with business rules such as checking for sufficient data before calculating the desired results.

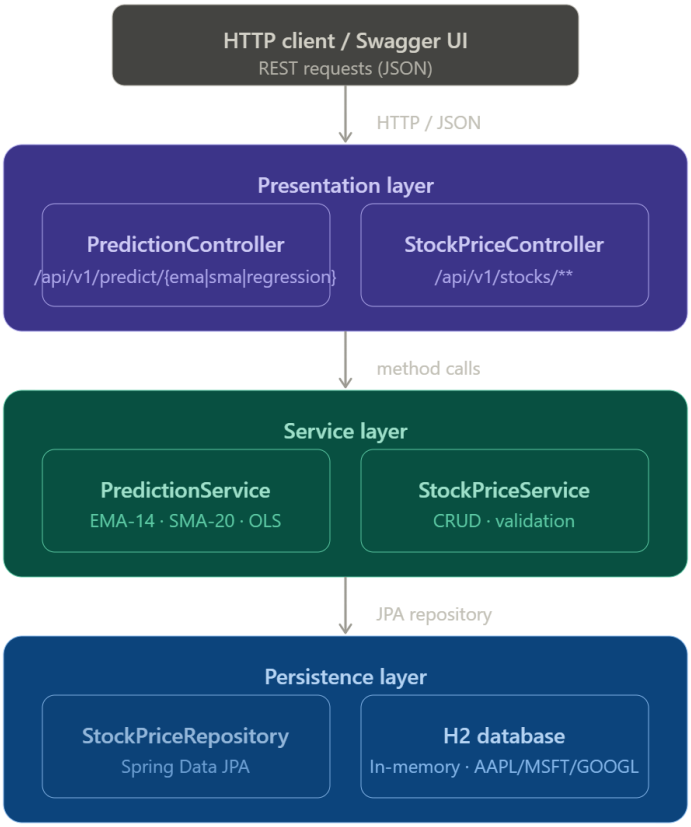
The persistence layer uses the Spring Data JPA library with an in-memory H2 database, a lean database ideal for this implementation. The DataInitializer automatically seeds 60 days of historical data for the three tickers so that the application starts, ensuring that the system is immediately functional.

CI/CD Pipeline

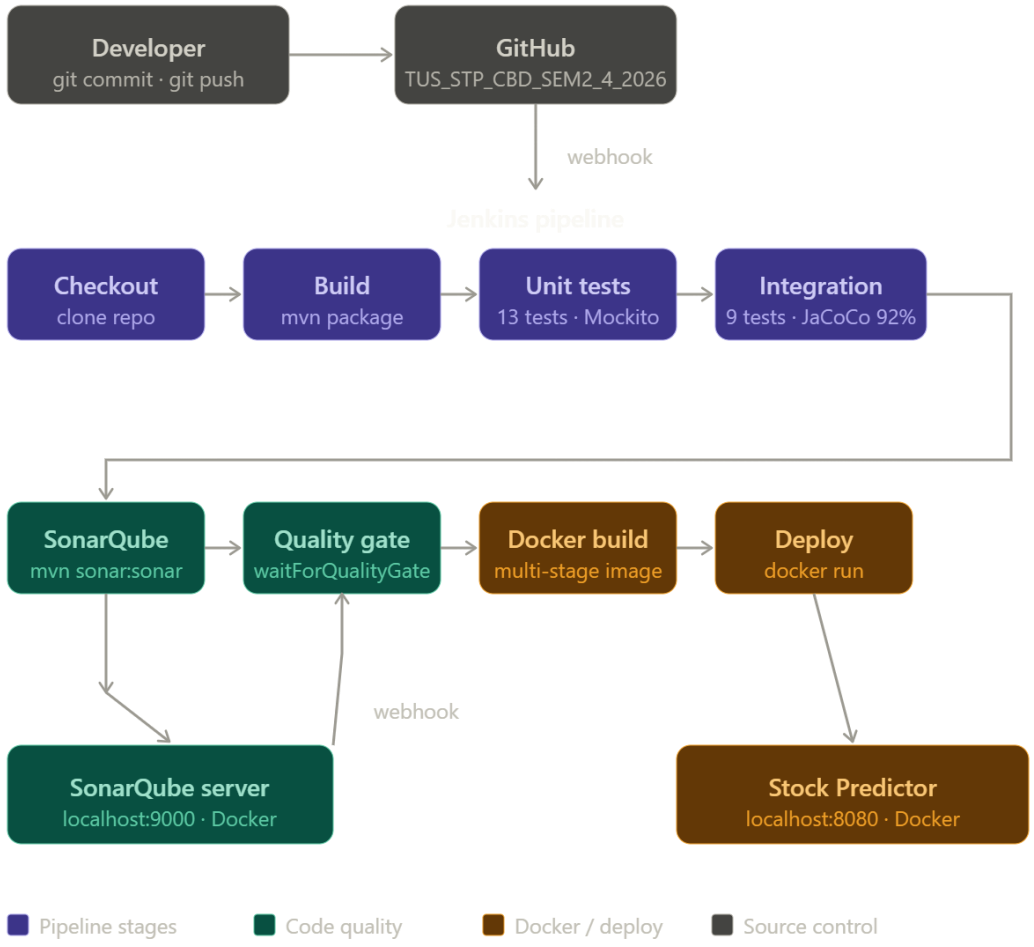
The project includes a complete Jenkins pipeline in Docker Compose with integrated SonarQube where the scripts are available along with the application. Each push to GitHub automatically triggers the checkout stages, Maven compilation, 22 automated tests (13 unit tests with Mockito, 9 integration tests with MockMvc), static code analysis in SonarQube with a quality gate covering at least 70% of the entire application, multi-stage Docker image build, and contain deployment.

Component Diagram

The following diagram illustrates the runtime components and their interactions:



CI/CD Pipeline Architecture



Alignment with the Test Pyramid

The testing strategy for this microservice is structured according to the Test Pyramid model introduced by Cohn (2009). The pyramid prescribes a large base of fast, isolated unit tests, a narrower middle band of integration tests, and an optional apex of end-to-end tests. This distribution maximises test coverage while minimising execution time and flakiness.

Level	Class	Tool	Count
Unit Tests (Base)	PredictionServiceTest	JUnit 5 + Mockito	10 tests
Integration Tests (Middle)	PredictionControllerIntegrationTest	MockMvc + Spring Boot Test	10 tests
End-to-End (Optional Apex)	Container smoke test in pipeline	curl / Docker HEALTHCHECK	1 check

Unit Tests - PredictionServiceTest

Unit tests operate in isolation by mocking the StockPriceRepository with Mockito. They verify the correctness of each time series algorithm independently of the database and the HTTP layer. Key scenarios covered include:

- EMA prediction returns a valid PredictionResult with the correct ticker symbol
- EMA confidence interval: confidenceLower is always less than confidenceUpper
- EMA forecast array length matches the requested forecast days parameter
- SMA prediction returns a result with the method field identifying SMA
- Linear regression prediction returns trend-based forecast points
- EMA and SMA raise IllegalArgumentException when fewer than 14 or 20 records exist
- EMA list size is mathematically correct relative to the window period
- SMA values are within the expected price range for constant input
- MAPE and RMSE are non-negative numeric values
- Standard deviation is positive for a set of varying prices

Integration Tests - PredictionControllerIntegrationTest

Integration tests load the full Spring application context using @SpringBootTest and exercise the HTTP layer via MockMvc. They validate end-to-end request processing including controller routing, service execution, JSON serialisation, and error handling. Key scenarios covered include:

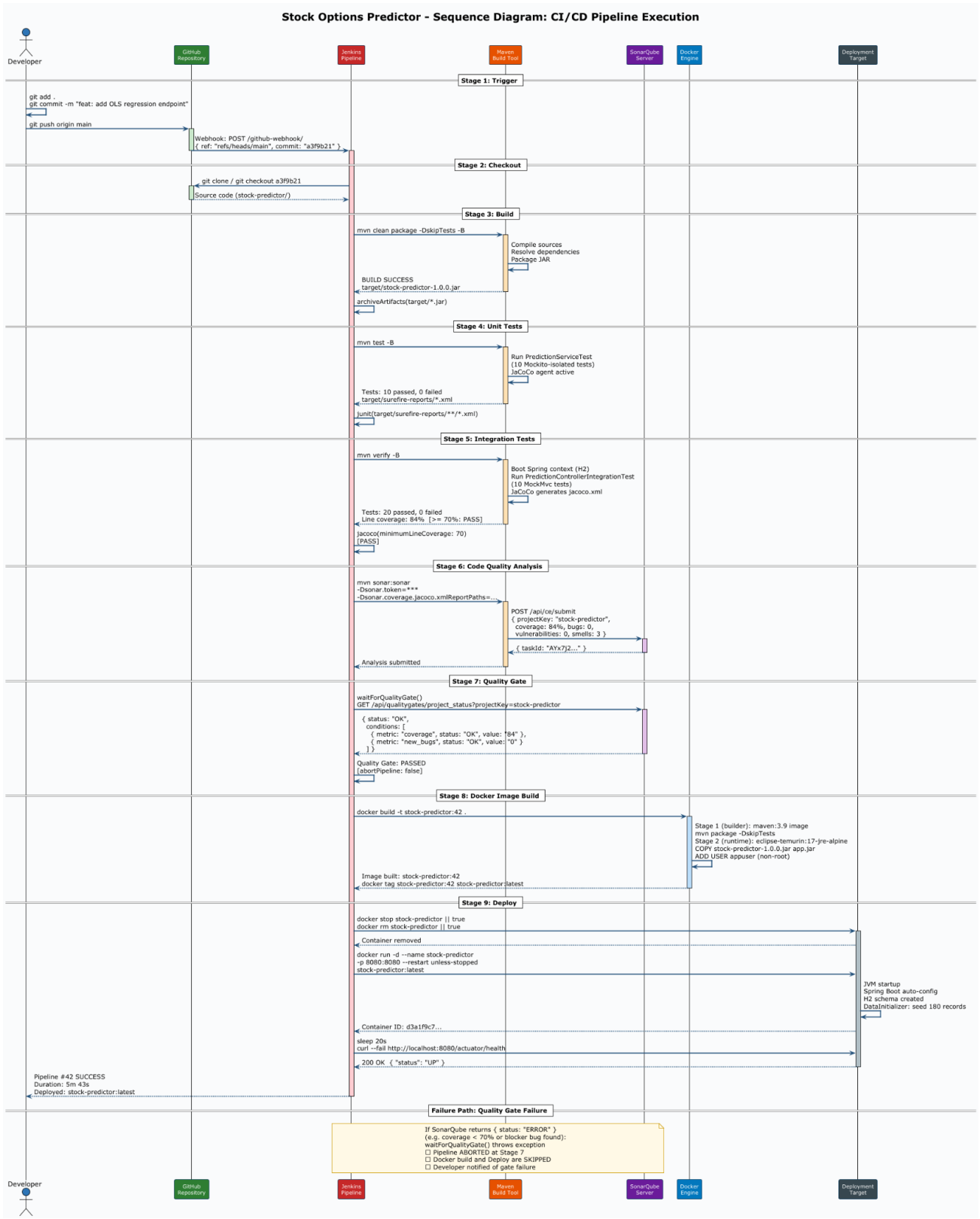
- GET /api/v1/predict/ema/AAPL returns HTTP 200 with a valid prediction JSON body
- GET /api/v1/predict/sma/MSFT returns HTTP 200 with SMA method identified
- GET /api/v1/predict/regression/GOOGL returns MAPE and RMSE as numeric fields
- GET /api/v1/predict/ema/UNKNOWN returns HTTP 400 for insufficient data
- GET /api/v1/stocks/tickers returns an array of available ticker strings
- GET /api/v1/stocks/AAPL returns historical price array with correct ticker
- POST /api/v1/stocks creates a new record and returns HTTP 201
- POST /api/v1/stocks returns HTTP 400 when required fields are missing
- GET /actuator/health returns HTTP 200 with status UP

Coverage Enforcement

JaCoCo is configured as a Maven plugin and enforces a minimum line coverage ratio of 70% as a quality gate. The Maven verify phase fails if coverage drops below this threshold. Coverage reports are generated in XML format (target/site/jacoco/jacoco.xml) for consumption by SonarQube. The DataInitializer and model classes are excluded from the coverage requirement to focus enforcement on testable business logic.

Pipeline Description

The entire application pipeline is defined within the Jenkinsfile located in the repository root and available on GitHub, as well as in a GitHub Actions workflow at **.github/workflows/ci-cd.yml**. Each step of the process is designed to fail quickly; if any step ends with a status code other than zero, the pipeline is aborted and all subsequent steps are skipped.


Stock Options Predictor - Sequence Diagram: CI/CD Pipeline Execution
Developer
GitHub Repository
Jenkins Pipeline
Maven Build Tool
SonarQube Server
Docker Engine
Deployment Target
Stage 1: Trigger
git add .
git commit -m "feat: add OLS regression endpoint"
git push origin main
Webhook: POST /github-webhook/
{ ref: "refs/heads/main", commit: "a3f9b21" }
Stage 2: Checkout
git clone / git checkout a3f9b21
Source code (stock-predictor/)
Stage 3: Build
mvn clean package -DskipTests -B
Compile sources
Resolve dependencies
Package JAR
BUILD SUCCESS
target/stock-predictor-1.0.0.jar
archiveArtifacts(target/*.jar)
Stage 4: Unit Tests
mvn test -B
Run PredictionServiceTest
(10 Mockito-isolated tests)
JaCoCo agent active
Tests: 10 passed, 0 failed
target/surefire-reports/*.xml
junit(target/surefire-reports/**/*.xml)
Stage 5: Integration Tests
mvn verify -B
Boot Spring context (H2)
Run PredictionControllerIntegrationTest
(10 MockMvc tests)
JaCoCo generates jacoco.xml
Tests: 20 passed, 0 failed
Line coverage: 84% [>= 70%: PASS]
jacoco(minimumLineCoverage: 70)
[PASS]
Stage 6: Code Quality Analysis
mvn sonar:sonar
-Dsonar.token=***
-Dsonar.coverage.jacoco.xmlReportPaths=...
POST /api/ce/submit
{ projectKey: "stock-predictor",
coverage: 84%, bugs: 0,
vulnerabilities: 0, smells: 3 }
{ taskId: "AYx7j2..." }
Analysis submitted
Stage 7: Quality Gate
waitForQualityGate()
GET /api/qualitygates/project_status?projectKey=stock-predictor
{ status: "OK",
conditions: [
{ metric: "coverage", status: "OK", value: "84" },
{ metric: "new_bugs", status: "OK", value: "0" }
] }
Quality Gate: PASSED
[abortPipeline: false]
Stage 8: Docker Image Build
docker build -t stock-predictor:42 .
Stage 1 (builder): maven:3.9 image
mvn package -DskipTests
Stage 2 (runtime): eclipse-temurin:17-jre-alpine
COPY stock-predictor-1.0.0.jar app.jar
ADD USER appuser (non-root)
Image built: stock-predictor:42
docker tag stock-predictor:42 stock-predictor:latest
Stage 9: Deploy
docker stop stock-predictor || true
docker rm stock-predictor || true
Container removed
docker run -d --name stock-predictor
-p 8080:8080 --restart unless-stopped
stock-predictor:latest
JVM startup
Spring Boot auto-config
H2 schema created
DataInitializer: seed 180 records
Container ID: d3a1f9c7...
sleep 20s
curl --fail http://localhost:8080/actuator/health
200 OK { "status": "UP" }
Pipeline #42 SUCCESS
Duration: 5m 43s
Deployed: stock-predictor:latest
Failure Path: Quality Gate Failure
If SonarQube returns { status: "ERROR" }
(e.g. coverage < 70% or blocker bug found):
waitForQualityGate() throws exception
☐ Pipeline ABORTED at Stage 7
☐ Docker build and Deploy are SKIPPED
☐ Developer notified of gate failure

Stage 1: Checkout

The entire pipeline process begins by cloning the GitHub repository using SCM credentials confirmed in Jenkins; the commit's SHA is then logged to ensure traceability. The commit SHA is logged to the console to ensure traceability between the screencast demonstration and the repository history. Key command: **git log --oneline -5**. The stage ensures that the workspace is always synchronised with the remote branch before any build activity commences.

Stage 2: Build

Maven compiles the source code and packages it into an executable JAR artifact using the spring-boot-maven-plugin. Tests are intentionally omitted at this stage to separate compilation from application verification. Main command: **mvn clean package -DskipTests -B**. The resulting JAR (target/stock-predictor-1.0.0.jar) is archived as a build artifact and identified for subsequent steps, as can be seen in the Jenkins deployment process.

Stage 3: Unit Tests

The Maven Surefire plugin is responsible for running JUnit 5 unit tests. These tests are isolated from the Spring context using Mockito mocks for the repository dependencies, with the main command being: **mvn test -B**. Test results are published using the Jenkins JUnit publisher, making pass/fail results visible in the pipeline dashboard. The stage stops the pipeline if any test fails.

Stage 4: Integration Tests

Spring Boot integration tests are executed using the Maven failsafe plugin via the verify lifecycle phase. The complete Spring application context is loaded using an H2 in-memory database, and HTTP requests are dispatched via MockMvc. Main command: **mvn verify -B**. JaCoCo code coverage data is collected during this phase and published to Jenkins. The pipeline fails if row coverage falls below 70%.

Stage 5: Code Quality Analysis - SonarQube

The Maven plugin SonarQube is responsible for analyzing the static of all source code as soon as it is invoked, primarily detecting bugs, identifying code problems, measuring duplication, and verifying critical security points. Main command: **mvn sonar:sonar -Dsonar.host.url=... -Dsonar.token=... -Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml**. The JaCoCo XML report is used by SonarQube to display test coverage directly in the source code and is presented in the SonarQube dashboard.

Stage 6: Quality Gate

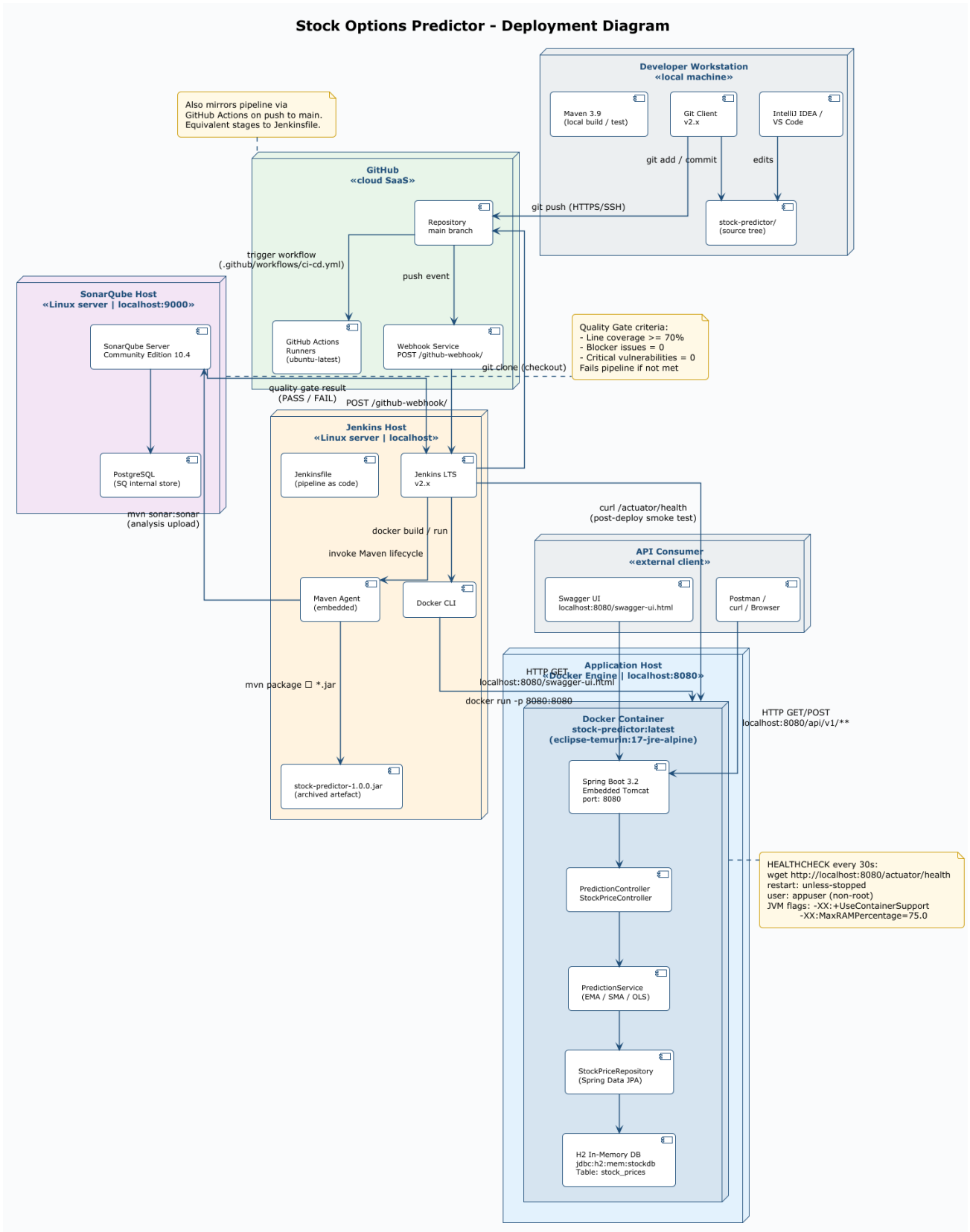
At this stage, the pipeline pauses and initiates a query to the SonarQube API to retrieve the Quality Gate status of the current analysis. The Quality Gate is configured for a minimum coverage of 70% of the lines, zero blocking issues, and zero critical vulnerabilities. Main command: **waitForQualityGate abortPipeline: true**. If the Quality Gate fails, the pipeline is aborted, and the Docker build step is never reached, preventing unqualified code from being containerized and stopping the entire pipeline.

Stage 7: Docker Image Build

The Dockerfile aimed to create a Docker image in several steps, the first being the Maven image, which complicates the application. The second creates a copy of the generated JAR for a lightweight Eclipse Temurin JRE Alpine image. Key commands: **docker build -t stock-predictor:${BUILD_NUMBER}** . and **docker tag stock-predictor:${BUILD_NUMBER} stock-predictor:latest**. A non-root user (**appuser**) is created in the image to follow the principle of least privilege.

Stage 8: Deploy

This is the final step and it stops and removes any container that is running with the same name as the application and starts a new container from a new image, with a 20-second window allowed before a health check is performed on **/actuator/health**. Main commands: **docker stop stock-predictor**, **docker run -d --name stock-predictor -p 8080:8080 stock-predictor:latest**, **curl –fail http://localhost:8080/actuator/health**. Ansible (deploy.yml) provides an equivalent idempotent deployment for infrastructure environments.



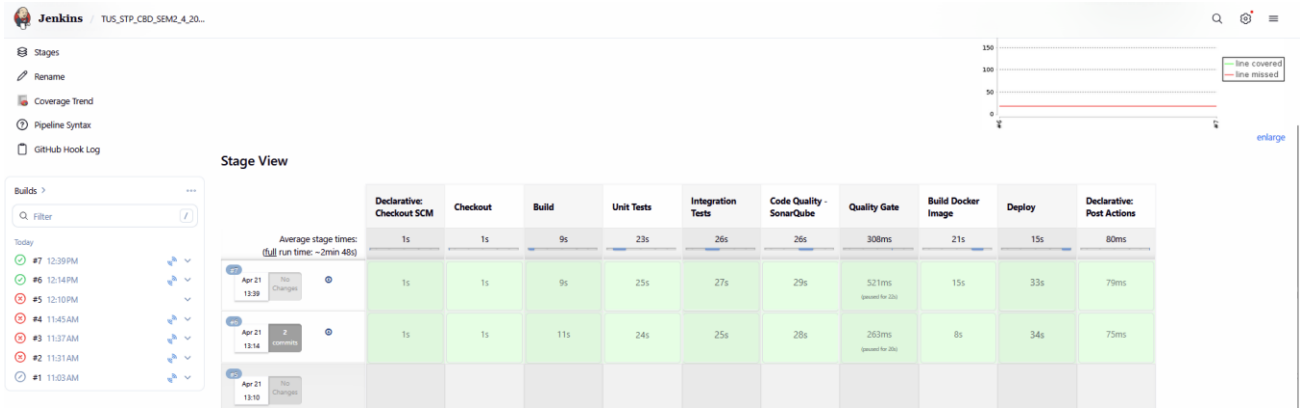
Pipeline Stage Summary

Stage	Key Command / Tool	Fail Condition	Artefact
Checkout	git checkout / scm	SCM error	Source tree
Build	mvn clean package -DskipTests	Compile error	*.jar
Unit Tests	mvn test	Test failure	Surefire XML
Integration Tests	mvn verify	Test failure / coverage < 70%	JaCoCo XML
SonarQube	mvn sonar:sonar	Analysis error	SQ report
Quality Gate	waitForQualityGate	Gate fails	None
Docker Build	docker build	Image build error	Docker image
Deploy	docker run + health check	Health check fails	Running container

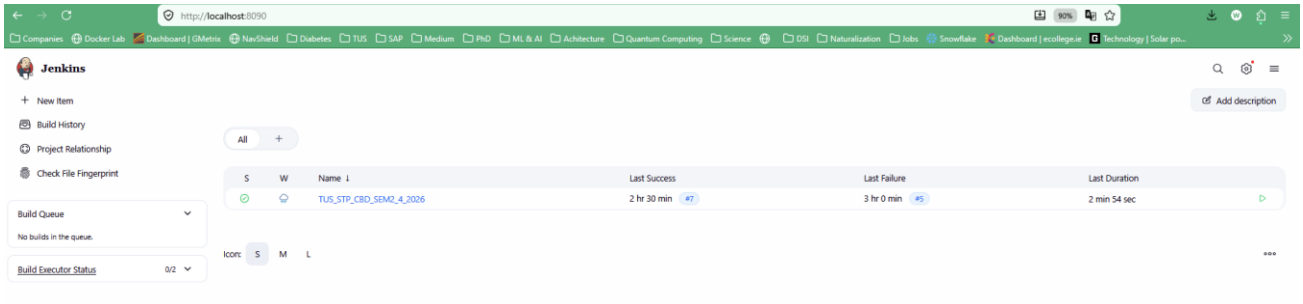
Evaluation and Reflection

Based on local machine benchmarks, the estimated end-to-end pipeline execution time is broken down below:

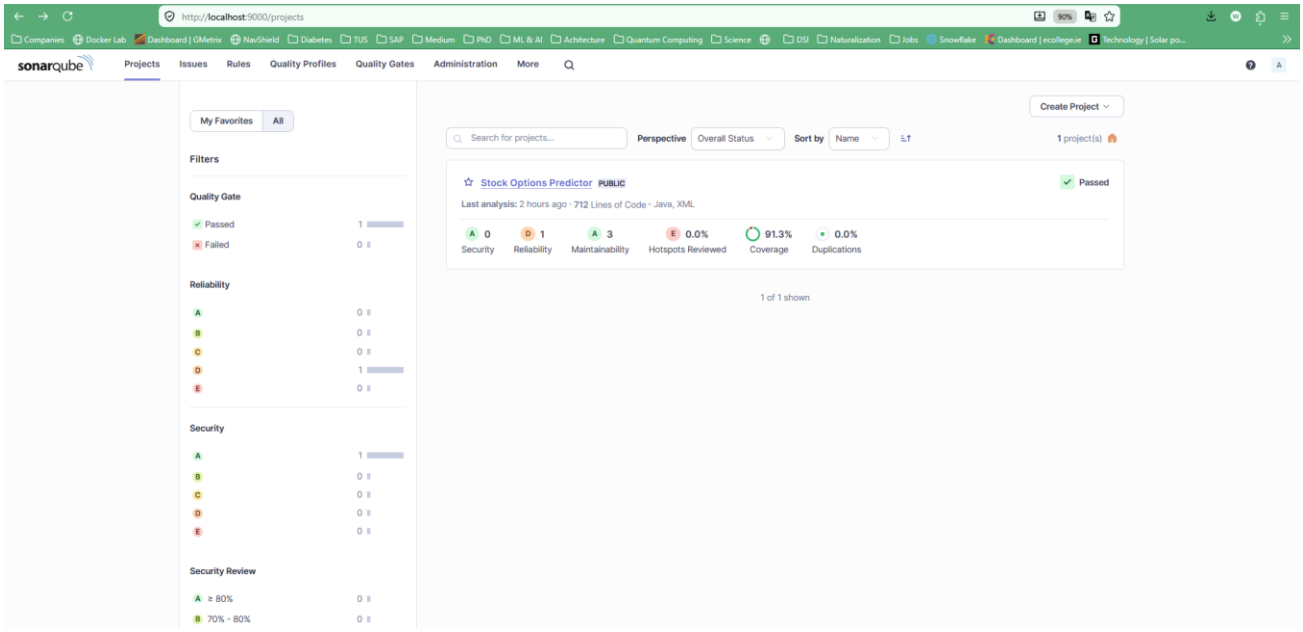
Stage	Estimated Duration	Notes
Checkout	1-10 seconds	Dependent on network and repo size
Build (mvn package)	10-90 seconds	First run downloads dependencies
Unit Tests	10-15 seconds	20 fast Mockito-isolated tests
Integration Tests	10-45 seconds	Spring context boot adds overhead
SonarQube Analysis	10-60 seconds	Network call to SonarQube server
Quality Gate check	60-60 seconds	API polling with up to 5 retries
Docker Build	10-120 seconds	Multi-stage, faster with layer cache
Deploy + health check	10-40 seconds	20s startup window + curl probe
Total (estimated)	~2 to 7 minutes	Varies with cache and hardware



The entire pipeline was executed with a high level of automation, with each step, from code submission to container execution, performed without human intervention. The only optional manual step would be approval for production.



SonarQube



Conscious Trade-Off: Speed vs. Coverage

A deliberate trade-off was made regarding the separation of unit and integration test phases. Running all tests in a single Maven command (mvn verify) would reduce total execution time by approximately 30 to 45 seconds because the Spring application context would only be loaded once. However, this approach conflates unit and integration test results, making it harder to identify at which level a failure occurred. The decision was made to prioritise diagnostic clarity over pipeline speed.

A second trade-off concerns the choice of H2 as the persistence store. H2 simplifies the pipeline significantly because no external database container must be provisioned, started, and health-checked before tests can run. The trade-off is that H2 is not representative of a production relational database, and subtle dialect differences (particularly around date functions and index behaviour) could allow bugs to pass the test suite that would be caught by a PostgreSQL or MySQL integration test environment.

Identified Limitations

- The H2 in-memory database is reset on each application restart, meaning historical data is not persisted between deployments. A production deployment would require an external database with a migration tool such as Flyway or Liquibase.
- The time series algorithms (EMA, SMA, OLS) are deterministic and do not account for volatility clustering or non-stationarity. More sophisticated models such as ARIMA or LSTM neural networks would produce more accurate financial forecasts.
- The SonarQube instance runs locally. In a team environment, a shared hosted instance (e.g., SonarCloud) would be required to centralise quality gate results across developers.
- The pipeline does not include a rollback mechanism. If the deployed container fails its health check after a period of use, the previous image is not automatically restored.

Suggested Improvements

- Replace H2 with PostgreSQL using Testcontainers for integration tests. Testcontainers spins up a real Docker-based PostgreSQL instance during the test phase, providing full dialect fidelity without requiring a persistent external database.
- Introduce a canary deployment strategy using Docker Swarm or Kubernetes. A percentage of traffic would be routed to the new container before the old one is terminated, reducing the risk of a failed deployment impacting all users.
- Add a performance testing stage using Gatling or k6. Load tests would run after integration tests and before the Docker build to detect regressions in response time before the image is published.
- Migrate to SonarCloud for hosted, zero-infrastructure code quality analysis with integration into GitHub pull request checks.
- Implement semantic versioning (SemVer) for Docker image tags based on Git tags, replacing the current BUILD_NUMBER approach. This would improve traceability between deployed images and source code commits.

Repository

Repository URL	https://github.com/wevertonsc/TUS_STP_CBD_SEM2_4_2026
Branch	main
Visibility	Public
Pipeline file	Jenkinsfile (root directory)
GitHub Actions	.github/workflows/ci-cd.yml
Dockerfile	Dockerfile (root directory)
Ansible playbook	ansible/deploy.yml
SonarQube config	sonar/sonar-project.properties

The repository contains all source code, the Jenkinsfile pipeline definition, the GitHub Actions workflow, the Dockerfile, the Ansible deployment playbook, and this report. The commit history reflects incremental development of each feature: data model, repository, service algorithms, REST controllers, tests, and CI/CD configuration. The screencast demonstration references a specific commit identifiable by its commit message in the repository history.

References

Fowler, M. (2006) Continuous Integration. Available at: https://martinfowler.com/articles/continuousIntegration.html (Accessed: 19 April 2025).

Humble, J. and Farley, D. (2010) Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley.

Forsgren, N., Humble, J. and Kim, G. (2018) Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press.

Kim, G., Behr, K. and Spafford, G. (2013) The Phoenix Project: A Novel About IT, DevOps, and Helping Your Business Win. Portland: IT Revolution Press.

SonarSource (2024) SonarQube Documentation. Available at: https://docs.sonarsource.com/sonarqube (Accessed: 19 April 2025).